Date:09/10/2026

9. Hyperparameter Optimization and Model Selection

For digits dataset in sklearn.dataset, try the following classifiers and find out the one that gives best performance. Also find the optimal parameters for that classifier.

Procedure:

Step 1: Import the required libraries and classifiers from Scikit-learn and Pandas.
Step 2: Load the Digits dataset using `load_digits()`.
Step 3: Store the input features in `X` and the target labels in `y`.
Step 4: Define the five classification models: SVM, Random Forest, Logistic Regression, Gaussian Naive Bayes, and Decision Tree.
Step 5: Specify the hyperparameter values to be tested for each classifier.
Step 6: Create an empty list to store the best scores and optimal parameters of each classifier.
Step 7: Apply `GridSearchCV` with five-fold cross-validation to each classifier.
Step 8: Fit each model to the Digits dataset and identify the best hyperparameter combination.
Step 9: Store the best cross-validation score and optimal parameters of each classifier in the results list.
Step 10: Convert the results list into a Pandas DataFrame.
Step 11: Sort the classifiers in descending order of their best cross-validation scores.
Step 12: Display the results and identify the classifier with the highest cross-validation score as the best-performing model.

Solution:

```
from sklearn.datasets import load_digits
from sklearn.model_selection import GridSearchCV
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
import pandas as pd

digits = load_digits()
X, y = digits.data, digits.target

models = {
    "SVM": {"model": Pipeline([("scaler", StandardScaler()), ("svc", SVC())]),
            "params": {"svc__C": [1, 10, 20], "svc__kernel": ["rbf", "linear"]}},
    "Random Forest": {"model": RandomForestClassifier(random_state=42),
                      "params": {"n_estimators": [50, 100, 200], "max_depth":
[None, 10, 20]}},
    "Logistic Regression": {"model": Pipeline([("scaler", StandardScaler()),
("lr", LogisticRegression(max_iter=5000))]),
                            "params": {"lr__C": [0.1, 1, 10]}},
    "GaussianNB": {"model": GaussianNB(),
                   "params": {"var_smoothing": [1e-9, 1e-8, 1e-7]}},
```

```
    "Decision Tree": {"model": DecisionTreeClassifier(random_state=42),
                      "params": {"criterion": ["gini", "entropy"], "max_depth":
[None, 10, 20]}}
}

scores = []

for name, mp in models.items():
    clf = GridSearchCV(mp["model"], mp["params"], cv=5)
    clf.fit(X, y)
    scores.append({"model": name, "best_score": clf.best_score_, "best_params":
clf.best_params_})

df = pd.DataFrame(scores, columns=["model", "best_score", "best_params"])
print(df.sort_values("best_score", ascending=False).to_string(index=False))
```

```
              model  best_score                                    best_params
                SVM    0.951023        {'svc__C': 10, 'svc__kernel': 'rbf'}
      Random Forest    0.941031    {'max_depth': None, 'n_estimators': 200}
Logistic Regression    0.925449                             {'lr__C': 0.1}
         GaussianNB    0.832518                  {'var_smoothing': 1e-07}
      Decision Tree    0.809698 {'criterion': 'entropy', 'max_depth': None}
```